



UNIVERSITY OF THE PHILIPPINES

College of Science

Department of Computer Science

Bachelor of Science in Computer Science

Leian Carl Dela Cruz

MaBayBay-OCR 2.0: A Context-Aware Baybayin Transliteration System

SP Adviser:

ROBERT R. ROXAS, PhD

Department of Computer Science

University of the Philippines Cebu

Date of Submission

May 2026

Permission is given for the following people to have access to this SP:

Available to the general public	No
Available only after consultation with author/SP adviser	No
Available only to those bound by confidentiality agreement	Yes

Student's signature:

Signature of the SP Adviser:

APPROVAL SHEET

The faculty of the University of the Philippines Cebu, Department of Computer Science approves the Special Problem entitled:

MaBaybay-OCR 2.0: A Context-Aware Baybayin Transliteration System

By Leian Carl Dela Cruz

ROBERT R. ROXAS

SP Adviser

DHARYLL PRINCE M. ABELLANA

Department Chair

Date Signed

Date Signed

A Special Problem
Presented to the Faculty of the
Department of Computer Science
University of the Philippines Cebu

MaBayBay-OCR 2.0: A Context-Aware Baybayin Transliteration System

In Partial Fulfillment of the Requirements of
Bachelor of Science in Computer Science

Leian Carl Dela Cruz

ROBERT R. ROXAS

Adviser

May 2026

Abstract

The resurgence of Baybayin, the pre-colonial Philippine writing system, in digital humanities has highlighted significant challenges in its automated transliteration. While Optical Character Recognition (OCR) systems can recognize Baybayin characters, they struggle with the script's inherent lexical ambiguity, where a single token can map to multiple Latin words. This study developed *MaBaybay-OCR 2.0*, a context-aware system that integrates a pre-trained RoBERTa-Tagalog model utilizing Masked Language Model (MLM) scoring via Pseudo-Log-Likelihood (PLL). To evaluate the system, a gold-standard dataset of 1,600 samples across 15 common ambiguous pairs was constructed. Experimental results demonstrated that the proposed system achieved an accuracy of 83%, a 37% improvement over the 46% accuracy of the baseline heuristic method. Furthermore, the system maintained an average runtime of 237 ms per sample, providing a superior balance of linguistic precision and computational efficiency for real-time applications. The study contributes a context-aware disambiguation methodology for *MaBaybay-OCR*, a specialized benchmark dataset for ambiguity resolution, and an empirical analysis of bidirectional contextual features, thereby advancing the digital preservation and revitalization of this cultural heritage.

Keywords: *Baybayin, Natural Language Processing, RoBERTa-Tagalog, Lexical Ambiguity*

<https://github.com/leidc024/MaBayBay-OCR 2.0>

Acknowledgments

Una sa lahat, nagpapasalamat ako sa Panginoong Maykapal, sa Kanyang walang sawang paggabay, pagbibigay ng lakas, at karunungan sa bawat hamon na aking hinarap habang isinasagawa ang pananaliksik na ito. Kung wala ang Kanyang patnubay, ang pag-aaral na ito ay hindi maisasakatuparan.

Taos-puso kong ipinapahayag ang aking pasasalamat sa aking tagapayo na si **Dr. ROBERT R. ROXAS**, para sa kanyang walang sawang paggabay, mahahalagang mungkahi, at suporta sa buong proseso ng pananaliksik. Ang kanyang kaalaman at malasakit ay naging malaking tulong upang mapabuti ang kalidad ng pag-aaral na ito.

Sa Unibersidad ng Pilipinas, sa pagbibigay sa akin ng pagkakataong makapag-aral sa isang prestihiyosong pamantasan. Naging mahalagang daluyan ito ng aking paghubog at paglago bilang isang mag-aaral at indibidwal, na aking babaunin at gagamitin sa aking paglalakbay sa hinaharap.

Sa aking mga kaibigan at sa aking kasintahan, na nagbigay ng tulong, inspirasyon, at moral na suporta sa mga panahong ako ay nahihirapan, maraming salamat sa inyong presensya at pag-unawa.

Pinakamahalaga, sa aking pamilya, sa kanilang walang sawang pagmamahal, sakripisyo, at paniniwala sa aking kakayahan. Ang kanilang suporta ang nagsilbing matibay na pundasyon upang aking maabot ang tagumpay na ito.

Sa lahat ng naging bahagi ng paglalakbay na ito—maliit man o malaki ang naitulong—taos-puso akong nagpapasalamat.

Contents

1	Introduction	1
1.1	Background of the Study	1
1.2	Statement of the Problem	3
1.3	Objectives	3
1.4	Scope and Limitations	4
1.5	Significance of the Study	5
2	Review of Related Literature	7
2.1	Baybayin Writing System	7
2.2	Current Context-Aware Baybayin Transliteration System	9
2.2.1	Challenges and Limitations	10
2.3	MaBayBay-OCR	11
2.4	Mask Language Model Scoring	12
2.4.1	Tagalog Language	13
2.4.2	RoBERTa-Tagalog	14
2.5	Statistics for NLP	15
3	Methodology	16
3.1	Analysis	16
3.2	Design	18
3.2.1	MaBayBay-OCR 2.0 System Architecture	18
3.2.2	Architectural Framework of the Disambiguation Layer	19
3.2.3	Ambiguous Pairs Candidate Generation	20

3.2.4	Standardized Evaluation Dataset Design	20
3.3	Implementation	21
3.3.1	Disambiguation Layer Integration to MaBaybay-OCR . . .	21
3.3.2	Pseudo-Log-Likelihood (PLL) Scoring for Context-Aware Disambiguation	23
3.3.2.1	PLL Computation	24
3.3.2.2	MLM Score Normalization	24
3.3.3	Frequency-Based Scoring for No-Context Scenarios . . .	25
3.3.3.1	Corpus Collection	25
3.3.3.2	Frequency Score Implementation	26
3.3.4	Baybayin Ambiguous Pairs	26
3.3.5	Standardized Evaluation Dataset	28
3.4	Testing	29
3.5	Evaluation	31
4	Results and Discussion	32
5	Conclusion and Future Work	35
	Appendices	41
Appendix A	MaBayBay-OCR Console	42
Appendix B	BaybayinDisambiguator Class	44
Appendix C	MLM-PLL Code	47
Appendix D	Frequency-Based Code	51
Appendix E	System Integration	52
Appendix F	Testing Framework	55
Appendix G	Ambiguous Words	60

List of Figures

2.1	Baybayin Characters Chart	8
2.2	Application of Kudlit and Krus-Kudlit in Baybayin.	8
3.1	MaBayBay OCR 2.0 pipeline	18
3.2	Architectural Framework of the Context-Aware Disambiguation Layer	19
3.3	Workflow for the Automated Extraction of Baybayin-Latin Am- biguous Pairs.	20
3.4	Disambiguation Integrated into MaBaybay-OCR	22
3.5	Code Snippet of the Scoring Process	23
3.6	Mathematical Equation for Candidate Pseudo-Log-Likelihood (PLL) Computation	24
3.7	Equation for Softmax Normalization of Masked Language Model Scores.	25
3.8	Formula for the Log-Normalized Frequency Scoring Mechanism	26
3.9	Sample Structure of the Finalized Ambiguous Pair Dataset . . .	27
3.10	Web-Based Interface for Ambiguous Pair Annotation and Validation	28
3.11	Sample Baybayin-rendered OCR input for the asero/asido pair. .	29
3.12	JSON structure used for representing transliteration test cases .	30
A.1	The MaBayBay-OCR Interface	42
A.2	Sample output of the system	43
B.1	Transformer Model Initialization	45

B.2	Default Configuration	46
C.1	Core PLL Computation	49
C.2	MLM Score Normalization	50
D.1	Frequency Scoring	51
E.1	MATLAB Integration Code	54
F.1	Sample Test Script	59

List of Tables

4.1	Resulting Accuracies per Method	32
4.2	Runtimes per method for one sample	33
G.1	List of Ambiguous Word Pairs Used in the Dataset	61

Chapter 1

Introduction

This chapter introduces the foundation of the study by presenting the background and context of the research. It defines the problem being addressed, states the objectives of the study, and discusses its scope and limitations. Additionally, this chapter explains the significance of the study and its potential contributions to the field.

1.1 Background of the Study

Baybayin is the ancient writing system of the Philippines. It is one of the most remarkable components of Filipino culture and serves as a symbol of cultural identity and heritage. As such, Baybayin can be regarded as a remembrance of the Philippines' deep historical roots and creative abilities. With increasing popularity among Filipinos, the script becomes a connection between the past and the present as it encourages the nation to take pride in its own heritage and feel its uniqueness while gaining recognition from other cultures for having an original writing system (Bayani Art, 2015; Camba, 2021). The name *Baybayin* comes from the Tagalog word "baybay," which means "to spell" (Narra Studio, 2019) .

The existence of Baybayin serves as a symbol of pre-colonial society and communication in 16th-century Philippines, prior to Spanish colonization (Lu,

2023). Studying and preservation of the Baybayin script contribute to honoring the cultural resilience of our ancestors, despite the script’s systematic decline under the influence of the Spanish Latin writing system (Stone, 2025).

Today, Baybayin has been increasingly apparent beyond internet spaces and into the actual physical environment as a form of artistic expression and cultural rebirth. It is now included in murals, public art, and pedestrian signage, showing its increasing revival. Additionally, Baybayin appears on Philippine passports, currency, and individuals familiar with the script may occasionally observe its use in public spaces and local establishments (Balbutin, 2023). Moreover, the Philippine government has launched attempts to further popularize Baybayin by inserting it into public signage, product labels, official documents, media, and the education system. This is further supported by House Bill 1022, also known as the National Writing System Act, proposed by the Committee on Basic Education and Culture of the Philippines in 2018, which aims to recognize Baybayin as the national writing system of the country (Tennant, 2024).

Recent studies on Baybayin have led to the development of Optical Character Recognition (OCR) systems that transliterate Baybayin text. Specifically, *MaBaybay-OCR*, a MATLAB-based Baybayin OCR package created by Rodney Pino, Renier Mendoza, and Rachelle Sambayan. This system is one of the early implementations of block-level Baybayin recognition. While it identifies Baybayin glyphs and allows for sentence transliteration, it only addresses surface-level recognition. It does not tackle the deeper linguistic challenges that come with the script.

For example, the Baybayin sentence ᜆᜄ᜔ ᜆᜄ᜔ ᜆᜄ᜔ ᜆᜄ᜔ ᜆᜄ᜔ ᜆᜄ᜔ ᜆᜄ᜔ can be transliterated as either “Ang buti ng tubig ay nasa lamesa” (“The good of water is on the table”) or “Ang bote ng tubig ay nasa lamesa” (“The bottle of water is on the table”). Given the surrounding context, the Baybayin word should be transliterated as *bote* instead of *buti*. However, the *MaBaybay-OCR* system is limited in generating all possible transliteration candidates and relies on sim-

plistic heuristics, such as selecting the first candidate, which frequently leads to semantically incorrect or nonsensical outputs (Pino et al., 2025).

Considering the presented problem, this research addresses the established limitations of *MaBaybay-OCR*—the pioneering software for block-level Baybayin transliteration. The specific objectives of this research are to: (1) implement a disambiguation layer, (2) generate a standard evaluation dataset specifically for Baybayin lexical disambiguation, and (3) develop a robust, context-aware Baybayin transliteration system.

1.2 Statement of the Problem

The rapid advancement of natural language processing (NLP) technologies has led to the development of optical character recognition (OCR) systems capable of transliterating Baybayin text into the Latin script. However, existing systems such as *MaBaybay-OCR* remain limited in handling the inherent linguistic ambiguities of Baybayin, particularly in resolving context-dependent transliterations. This limitation hinders the effectiveness, reliability, and broader adoption of Baybayin OCR systems, thereby affecting the accuracy of digital preservation and interpretation of the script. To address these gaps, this study aims to develop a context-aware Baybayin transliteration system, along with a standardized evaluation dataset, to improve transliteration accuracy and resolve ambiguity in Baybayin text.

1.3 Objectives

In order to address the problem mentioned above, this study aims to:

- Generate a standardized evaluation dataset specifically designed for testing Baybayin semantic disambiguation;

- Implement a disambiguation module that utilizes a pre-trained RoBERTa-Tagalog for masked language model (MLM) scoring via Pseudo Log Likelihood (PLL), and frequency statistical scoring.;
- Develop a context-aware, Baybayin transliteration system that integrates the disambiguation module to refine MaBayBay-OCR outputs; and
- Evaluate the accuracy and performance of the proposed system in resolving ambiguities compared to baseline heuristic methods. .

1.4 Scope and Limitations

This study aims to develop a post-processing module focused exclusively on the lexical disambiguation of Baybayin-to-Latin transliteration of MaBaybay-OCR. To achieve this, the system builds upon the transliterated output of the original system. The method of investigation employs a multi-method approach that combines a pre-trained RoBERTa-Tagalog model for masked language model scoring via pseudo-log-likelihood (MLM-PLL) and statistical features. This approach is designed to resolve both contextual semantic ambiguities in full sentences and zero-context spelling ambiguities in single, isolated words.

The limitations of the study are as follows:

1. The proposed system does not attempt to improve the character-level recognition accuracy of the base MaBayBay-OCR system;
2. The text processing is restricted to Filipino (Tagalog-based) text and does not extend to other Philippine languages that may also be written in Baybayin;
3. The scope of the disambiguation is limited to resolving specific types of orthographic ambiguity, namely interchanges between e/i, o/u, and d/r;

4. The system's contextual understanding is strictly bound by the foundational Tagalog corpora used to train the original RoBERTa-Tagalog model, while its spelling standardization relies entirely on the utilized formal frequency scoring and
5. The transliteration accuracy may be negatively affected by extremely colloquial text, "text-speak," or code-switching (Taglish) that falls outside standard Filipino orthography. .

1.5 Significance of the Study

This study aims to provide a robust technological solution to the semantic and orthographic challenges inherent in Baybayin transliteration. By successfully developing a context-aware system, the results of this research will be highly beneficial to the following sectors:

- **Natural Language Processing (NLP) Researchers and Developers:** Context-aware Baybayin transliteration systems remains a highly specialized and underexplored niche in computational linguistics. This study builds upon the pioneering block-level optical character recognition (OCR) software introduced in 2025. By proposing a novel method that integrates a masked language model (MLM) scoring, and lexical frequency scoring, this research establishes a highly robust framework for semantic and orthographic Baybayin disambiguation. Furthermore, this study contributes a newly generated, standardized evaluation dataset designed to rigorously test transliteration accuracy, speed , and this benchmark provides a vital technical baseline for future developers seeking to advance this specific domain
- **Linguists, Historians, and Cultural Preservationists:** This proposed system ensures that digitized documents in Baybayin are linguistically ac-

curate in addressing any mistakes in context and orthography. Consequently, this reduces the possibility of a semantic distortion when transliterating Baybayin texts. This ensures that linguists and historians are able to preserve information without being hampered by simplistic OCR errors.

- **Educators and Students:** The development of an accurate post-processing module creates a reliable technological tool for the academic sector. By producing semantically correct Latin outputs rather than first candidate selection from all possible candidates, the system prevents the propagation of misspelled words or incorrect heteronym translations among learners, and it facilitates a more effective and frictionless learning experience for individuals studying the script.

Chapter 2

Review of Related Literature

This chapter examines the current state of context-aware Baybayin transliteration, including its associated challenges, emerging OCR technologies, and issues related to lexical disambiguation. It also explores existing and potential approaches to addressing these limitations.

2.1 Baybayin Writing System

Baybayin is an abugida script, meaning each consonant symbol has an inherent vowel sound, which by default is /a/. Unlike the Latin script, Baybayin often combines consonants with vowels into a single character representation.

The script is composed of three standalone vowels and fourteen basic consonant characters, as shown in Figure 2.1. Each consonant character automatically represents a syllable ending in the vowel /a/. For example, the symbol for *ba* naturally reads as *ba*, while the symbol for *ka* reads as *ka*.

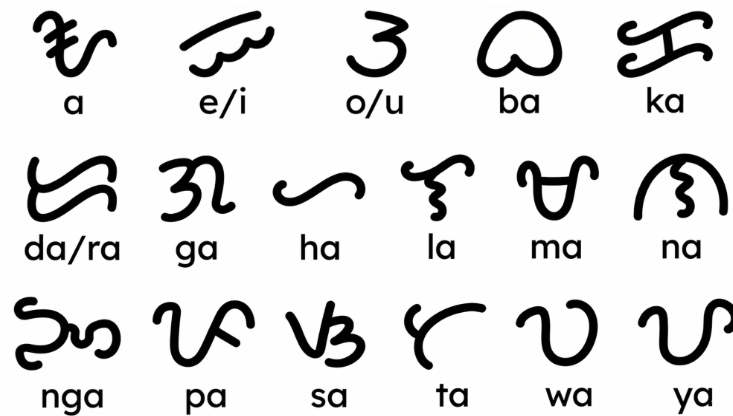


Figure 2.1. Baybayin Characters Chart

Note. Adapted from “What is Baybayin?” (Kuya Bai, 2022).

In order to modify the sound of the *katinig* (consonant) characters, a *kudlit* (diacritic) is added above the character to represent the /i/ or /e/ sounds and below the character to represent the /o/ or /u/ sounds. Moreover, a *krus-kudlit*, a cross-shaped symbol added beneath a consonant to cancel its inherent vowel and produce a standalone consonant, is illustrated in Figure 2.2. Baybayin is read and written from left to right, with spaces used to separate words. It is also important to note that, since the script is phonetic, words are generally written according to their *baybay* (pronunciation) rather than standardized spelling conventions.



Figure 2.2. Application of Kudlit and Krus-Kudlit in Baybayin.

Note. Adapted from “What is Baybayin?” (Kuya Bai, 2022).

2.2 Current Context-Aware Baybayin Transliteration System

Baybayin is difficult to transliterate accurately into the Latin script because of ambiguities in its vowel system, particularly the interchangeable use of e/i, o/u, and d/r. These limitations were among the reasons Spanish colonizers promoted their own alphabet, which contributed to Baybayin’s decline in the Philippines by the late seventeenth century (Pino et al., 2025).

A study by Bernardo and Estuar (2025) proposed one of the earliest solutions to this problem through *bAI-bAI*, a context-aware system capable of identifying the correct transliteration of ambiguous Baybayin words using multiple approaches, including word embeddings (WE), WE combined with part-of-speech (POS) tagging, and Large Language Model (LLM)-based methods. Their findings highlight a significant trade-off between accuracy and computational efficiency. The LLM-based approach, specifically using GPT-4o mini, achieved the highest accuracy of 90.52% but incurred a substantial runtime cost of 3280 ms per sample. In contrast, the word embedding-based approach demonstrated significantly faster performance at 5.35 ms per sample, but with lower accuracy at 77.46%.

However, despite recent developments in Baybayin optical character recognition (OCR) systems, there remains a lack of studies addressing transliteration ambiguity. More broadly, even within the field of natural language processing (NLP), literature on systems capable of handling multiple context-dependent transliterations remains limited, highlighting the niche yet significant nature of research in this area (Bernardo and Estuar, 2025).

The performance of the *bAI-bAI* system is evaluated using accuracy and runtime metrics, as the disambiguation task is not a binary classification problem but rather a selection task in which one correct candidate is chosen from multiple possible options for each ambiguous position. This task relies on exact

string matching, meaning that the generated output must precisely match the correct word (Bernardo and Estuar, 2025). The present study addresses these limitations, particularly the lack of research on Baybayin transliteration ambiguity.

2.2.1 Challenges and Limitations

According to Bernardo and Estuar (2025), the transliteration process is commonly performed in two main steps: character-level classification followed by word-level transliteration. They noted that some studies focus only on the classification stage, such as (Ligsay et al., 2022), which used a convolutional neural network (CNN) based on YOLOv3 and achieved 98.92% accuracy, the highest among CNN architecture methods (Pino et.al, 2025). Meanwhile, other studies extend their work by proposing their own methods for transliterating Baybayin words at the word level, including those by Pino et al. (2021) and Vilvar et al. (2022).

Word transliteration is carried out by assembling the outputs obtained from classifying each character. This process begins by breaking a Baybayin word into its individual symbols. Each symbol is then translated into its corresponding Latin form, and the resulting characters are combined to produce the final transliterated word (Pino et al., 2021). Sentences are a group of words that convey meaning. In order to create a context-aware Baybayin transliteration, implementing sentence-level transliteration is a necessary step. The bAI-bAI system builds upon the word recognition and transliteration approach of Vilvar et al. (2022), and applies Levenshtein distance to compare the predicted word with entries in a corpus to determine the final output (Bernardo and Estuar, 2025).

However, the original method does not include a mechanism for detecting word boundaries. To address this limitation, Bernardo and Estuar (2025) modified the horizontal dilation value during image preprocessing. This allows the

system to separate text into word-level segments. For each detected word block, the original word-level transliteration algorithm is used. The resulting words are combined, separated with spaces to create the final output sentence. A key limitation of this approach is that it only allows *bAI-bAI* to process Baybayin text arranged in a single horizontal line. Because the algorithm groups characters based only on their x-coordinates and does not consider their y-coordinates, it is unable to handle multi-line inputs.

Beyond *bAI-bAI*, there are limited existing systems that can determine whether a word should have an /i/ instead of an /e/, /u/ instead of an /o/, or /r/ instead of /d/ in Baybayin. Despite being one of the first attempts at sentence-level transliteration, it cannot effectively handle complex sentence-level Baybayin input due to its x-coordinate limitation. Most historical Baybayin documents have multi-line inputs, as shown in preserved Baybayin documents (University of Santo Tomas, 2021). Therefore, this study addresses this problem by using MaBaybay-OCR, the first text block-level Baybayin recognition system that can transliterate sentence-level inputs (Pino et al., 2025). Adding context-aware capability for this system is the primary goal of this research, leading to the development of MaBaybay-OCR 2.0.

2.3 MaBayBay-OCR

MaBaybay-OCR is a MATLAB-based optical character recognition (OCR) system designed to automatically transliterate the ancient Filipino Baybayin script into modern Latin text (Pino et al., 2025). The system employs Support Vector Machines (SVM), adapted from the authors' earlier work (Pino et al., 2021), to accurately classify characters and distinguish between Baybayin and standard Latin words within the same image.

The development of MaBaybay-OCR represents a significant step toward the digital preservation of the Baybayin script. As an improvement over the au-

thors' previous work, the system allows users to select between different OCR modes, enabling recognition of either purely Baybayin text or mixed Baybayin and Filipino Latin text within the same frame. This functionality provides valuable insights into pre-colonial Filipino writing systems and supports historical and archival research involving Baybayin (Pino et al., 2025).

A major limitation identified in the study is transliteration ambiguity. Since a single Baybayin symbol can represent multiple vowel sounds (e/i or o/u) and consonants (d/r), the system generates multiple possible Latin translations without considering semantic context.

This limitation highlights a gap in context-aware transliteration. Thus, the present study builds upon MaBaybay-OCR by incorporating a disambiguation mechanism to resolve such ambiguities and determine the most semantically appropriate translation. Additionally, it considers handling misspellings in single-word inputs.

2.4 Mask Language Model Scoring

A Masked Language Model (MLM), such as RoBERTa, is a neural network architecture trained by masking specific tokens within a sentence and requiring the model to predict the missing words based on the surrounding context (Liu et al., 2019). Unlike traditional autoregressive models that process text in a strictly left-to-right sequence, MLMs are inherently bidirectional, allowing them to consider both preceding and succeeding words when interpreting a token's semantic and grammatical role (Devlin et al., 2019). This bidirectional capability makes MLMs effective for evaluating the linguistic acceptability and overall coherence of a sentence.

The evaluation of sentence context using an MLM can be performed through Pseudo-Log-Likelihood (PLL) scoring. As described by Salazar et al. (2020), PLL is computed by masking each token in a sentence individually and mea-

asuring the model’s likelihood of correctly predicting the masked word given the remaining context. This approach enables sentence scoring without the left-to-right bias associated with autoregressive models. Prior work has demonstrated that PLL-based scoring is effective for rescoring tasks, where multiple candidate outputs are ranked to identify the most grammatically and contextually appropriate sentence (Salazar et al., 2020).

This methodology addresses a key limitation in existing Baybayin research. Existing LLM approaches incur high computational costs, as observed in GPT-based prompting approaches used in the *bAI-bAI* system (Bernardo and Estuar, 2025). By applying PLL scoring to a RoBERTa-based Tagalog model, this study proposes an efficient and context-aware disambiguation mechanism. Instead of relying on computationally expensive generative methods, the system ranks N-best candidate transliterations produced by the OCR. This enables the selection of the output that best aligns with the bidirectional semantic context of the Tagalog language, thereby supporting the development of an improved MaBaybay-OCR system.

2.4.1 Tagalog Language

Tagalog is the primary basis of Filipino, the national language of the Philippines, and is classified as an institutional language. It is categorized as a Central Philippine language within the Malayo-Polynesian branch of the Austronesian language family. Tagalog exhibits close linguistic relationships with Bikol, Bisayan, and Mansakan languages. Its origin is believed to be in either the eastern Visayas or northeastern Mindanao, with historical migration leading to its establishment in Southern Luzon, particularly in Manila and its surrounding provinces. The language subsequently expanded across the archipelago, comprising approximately a quarter of the Philippine population by 1940 and becoming a majority language by 1970. At present, Tagalog is spoken as either a first or second language by over 90% of Filipinos (Schachter and Reid,

2018). Moreover, Baybayin was historically used to write Tagalog, which serves as the primary focus of this study.

Tagalog possesses several linguistic characteristics that contribute to the complexity of Baybayin transliteration. Since Baybayin is an abugida with limited vowel representation, ambiguities commonly arise from the interchangeable use of vowels such as *e/i* and *o/u*, as well as consonants such as *d/r*. These ambiguities often require contextual understanding to determine the correct Latin-script transliteration. In addition, Tagalog is morphologically rich and highly contextual, where the meaning of words may vary depending on affixation, sentence structure, and surrounding words. As a result, accurate Baybayin transliteration cannot rely solely on character-level conversion and instead requires contextual and semantic analysis to produce meaningful outputs.

2.4.2 RoBERTa-Tagalog

To address the unique needs of Filipino NLP, Cruz and Cheng (2022) developed Tagalog-specific RoBERTa models pretrained on the TLUnified dataset. TLUnified is a large-scale and topically diverse pretraining corpus designed to overcome the limitations of previous datasets such as WikiText-TL-39, which were often too small or domain-specific to produce robust language models. These Tagalog RoBERTa models follow the robust optimization techniques introduced by Liu et al. (2019) to improve training efficiency and downstream task performance.

The Tagalog RoBERTa models are available in two primary configurations: a Base variant with 110 million parameters and a Large variant with 330 million parameters. Both models utilize a Byte-Pair Encoding (BPE) tokenizer trained on the TLUnified dataset with a vocabulary size of 32,000 subwords to ensure broad coverage of the morphological variations present in the Tagalog language. The models achieved state-of-the-art performance across several Filipino NLP benchmarks, including sentiment analysis using the Hatespeech

dataset, multiclass classification using the Dengue dataset, and natural language inference using NewsPH-NLI.

In the context of this study, RoBERTa-Tagalog serves as the primary language model for contextual scoring and ambiguity resolution in Baybayin transliteration. Its ability to capture contextual semantic relationships in Tagalog makes it suitable for resolving ambiguities that cannot be addressed through direct character-level transliteration alone.

2.5 Statistics for NLP

In the absence of surrounding sentence context, statistical features such as word frequency become essential for resolving script ambiguities. In standard Natural Language Processing (NLP) pipelines, word frequency functions as a descriptive statistical feature that reflects lexical distributions within a language corpus. For the proposed MaBaybay-OCR 2.0 system, applying word frequency as a statistical prior helps address common OCR misspellings, such as distinguishing between the valid word “doktor” and a non-existent variant like “doktod” by identifying the more probable intended token.

Empirical studies on OCR error correction have shown that incorporating frequency-based priors can reduce word error rates (WER) by up to 35% compared to systems that rely solely on character-level recognition (Kissos and Dershowitz, 2016). Furthermore, although Masked Language Models (MLMs) such as RoBERTa inherently capture certain frequency patterns during training, explicit frequency weighting remains important for mitigating the inflated scores that bidirectional models may assign to rare or out-of-vocabulary (OOV) sub-word fragments (Kauf and Ivanova, 2023).

In this study, statistical features complement the contextual predictions generated by the RoBERTa-Tagalog model, particularly in cases where isolated words lack sufficient semantic context for reliable spelling ambiguity resolution.

Chapter 3

Methodology

This chapter outlines the five primary phases involved in developing the context-aware Baybayin transliteration system: analysis, design, implementation, testing, and evaluation. The design and implementation phases are further organized into subsections that correspond to their respective objectives and overall purpose.

3.1 Analysis

The identification of the research problem began with an examination of how MaBaybay-OCR handled block-level text. It was observed that while the system was a pioneer in block-level recognition, its output was limited by its reliance on simplistic heuristics. Specifically, the system selects the first available candidate from a list of possible transliterations, which frequently results in semantically incorrect or nonsensical sentences. The analysis revealed that character-level accuracy, though high in existing Support Vector Machine (SVM) models, did not guarantee a readable or accurate transliteration of the intended message.

A secondary subproblem identified during this phase was the inherent orthographic ambiguity of the Baybayin script itself. Because Baybayin is an abugida script, the markers (*kudlits*) used to modify consonants represent multiple phonetic values in the modern Latin alphabet. The analysis focused on three pri-

many areas of overlap:

1. The interchangeability of the vowels /e/ and /i/.
2. The interchangeability of the vowels /o/ and /u/.
3. The dual phonetic representation of the character for /d/ and /r/.

By analyzing these overlaps, it was found that a major gap in existing literature was the absence of a post-processing disambiguation mechanism capable of resolving these specific “one-to-many” mappings based on semantic context.

Furthermore, the analysis phase scrutinized the computational trade-offs observed in the bAI-bAI system. While large language models (LLMs) such as GPT-4o mini achieved high accuracy in disambiguation (90.52%), they incurred a significant runtime cost of 3280 ms per sample. Conversely, faster methods such as word embeddings demonstrated lower accuracy (77.46%). This identified a clear gap: the need for a middle-ground solution capable of providing high linguistic depth without the extreme latency associated with generative LLMs. This finding directed the research toward the use of Masked Language Model scoring via Pseudo-Log-Likelihood (MLM-PLL) as a more computationally efficient alternative.

Finally, the analysis of the research landscape highlighted a critical lack of standardized testing data. It was found that most existing Baybayin datasets were optimized for character recognition rather than lexical disambiguation. This led to the identification of a procedural gap: the requirement to generate a new standardized evaluation dataset specifically designed for testing orthographic and semantic disambiguation in Baybayin sentences.

By synthesizing these findings, the analysis phase concluded that the development of a Baybayin context-aware system required three distinct components: a disambiguation layer using RoBERTa-Tagalog, the integration of statistical frequency features to handle zero-context words, and a benchmark

dataset for accuracy and performance validation. These requirements served as the foundation for the subsequent design and implementation phases.

3.2 Design

This section focused on constructing a conceptual framework to resolve the transliteration ambiguities identified in the analysis of the baseline MaBaybay-OCR system. The following subsections detail the system's architecture, the candidate generation logic, the scoring mechanism, and the criteria for the standardized evaluation dataset.

3.2.1 MaBayBay-OCR 2.0 System Architecture

The system was designed to have a modular post-processing layer that intercepts the raw output of the MaBaybay-OCR before the final Latin text is rendered. This architectural approach was chosen to maintain the integrity of the original character recognition while introducing a semantic filtering mechanism to resolve e/i, o/u, and d/r ambiguities. Figure 3.1 illustrates the proposed workflow of the system. The design follows a sequential pipeline consisting of image-to-candidate generation, disambiguation, and final candidate selection

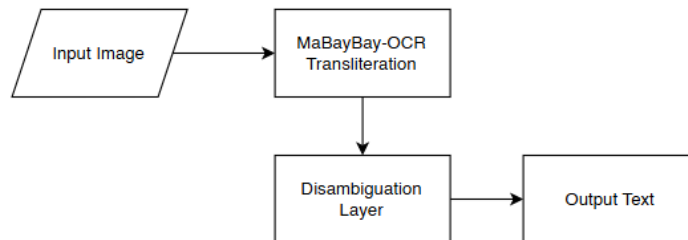


Figure 3.1. MaBayBay OCR 2.0 pipeline

3.2.2 Architectural Framework of the Disambiguation Layer

The disambiguation layer was integrated into the MaBaybay-OCR pipeline as a post-processing step that operates after optical character recognition. The MaBaybay-OCR system, originally implemented in MATLAB, produces an array cell output containing candidate transliterations for each position in the Baybayin text. These candidates are structured as either single-element arrays (unambiguous words where OCR confidence is high) or multi-element arrays (ambiguous positions with multiple plausible transliterations).

Figure 3.2 shows the framework used to ensure that the system can handle both full-sentence contexts and isolated-word scenarios. For sentences, the module was designed to utilize a bidirectional Masked Language Model (MLM), whereas for isolated words, it was designed to default to a statistical frequency check. This dual-path design addresses the limitation of the original MaBaybay-OCR system, which relies on a baseline heuristic.

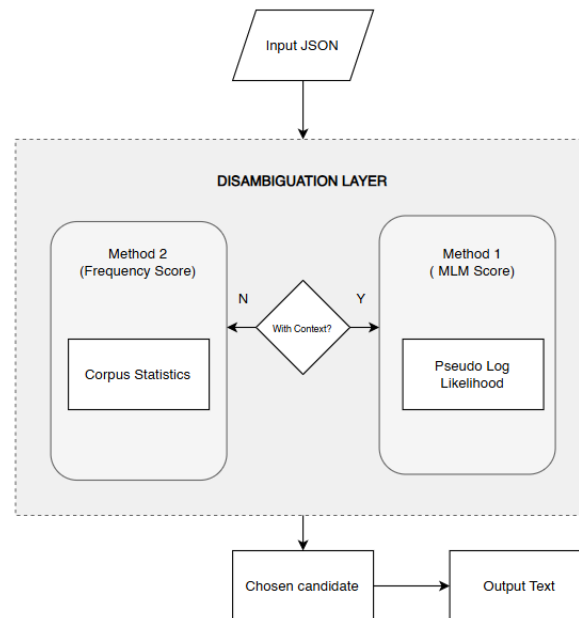


Figure 3.2. Architectural Framework of the Context-Aware Disambiguation Layer

3.2.3 Ambiguous Pairs Candidate Generation

The candidate generation phase was designed to systematically identify and map the inherent phonetic overlaps in the Baybayin script. The approach for this component involved the utilization of a comprehensive Tagalog corpus recognized by the MaBaybay-OCR system. A design was established to use a Python script that would process the corpus and convert each word into its Baybayin equivalent to find all possible ambiguous pairs.

This process focused on identifying word sets that result in identical Baybayin representations due to the shared characters for e/i, o/u, and d/r. The resulting mappings were designed to be stored in a CSV file. This process is illustrated in Figure 3.3.

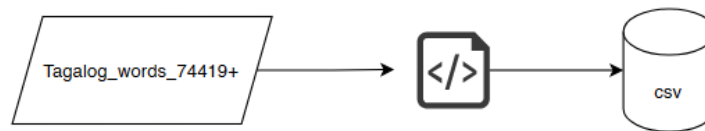


Figure 3.3. Workflow for the Automated Extraction of Baybayin-Latin Ambiguous Pairs.

3.2.4 Standardized Evaluation Dataset Design

To ensure the statistical validity of the system's performance, a standardized evaluation dataset was designed. The researcher identified the 15 most common ambiguous pairs from the generated CSV file to serve as the core benchmark. For each ambiguous set, the goal was to curate 50 unique sentences for every possible sense (meaning) of the word.

The dataset design follows the formula: **n pairs** $\times$ **m meanings each** $\times$ **50 sentences** = **p total sentences**. Most entries consist of ambiguous word pairs with two possible meanings. However, certain cases required expanded semantic modeling. For example, the *tuyo/toyo* entry was modeled using three distinct

sense interpretations due to contextual variations in usage. Similarly, the triplet *turo/todo/toro* contributed 150 sentences because of its three-way ambiguity. Overall, the finalized dataset contains 1,600 annotated sentences across all ambiguous sets. The full dictionary meanings of the ambiguous words are presented in Appendix G.1

By allocating 50 sentences per sense, the evaluation dataset provides a sufficient sample size for statistical analysis. This design helps demonstrate that the system’s correct predictions are driven by meaningful contextual distinctions rather than random chance. Furthermore, this standardized evaluation framework addresses the lack of benchmark datasets for Baybayin lexical disambiguation identified in previous studies.

3.3 Implementation

This section narrates the practical execution of the designs previously described.

3.3.1 Disambiguation Layer Integration to MaBaybay-OCR

The integration employs a Python-based disambiguation engine, titled BaybayinDisambiguator, invoked from MATLAB through a system call. Since MaBaybay-OCR natively represents transliteration outputs as MATLAB cell arrays, a wrapper script is introduced to transform these outputs into a JSON format compatible with the Python environment.

The overall workflow proceeds as follows: (1) *MaBaybay-OCR* generates up to three transliteration candidates per word position as a MATLAB cell array; (2) the wrapper script, `disambiguate_candidates.m`, serializes these candidates into JSON; (3) a Python entry-point script ingests the serialized data and applies the disambiguation engine, which scores each candidate using either contextual (MLM-PLL) or frequency-based features; and (4) the script returns the disambiguated sentence via standard output (`stdout`), which is subsequently

captured and processed within MATLAB.

A sample execution output of disambiguated text, as displayed in the MATLAB terminal, is shown in Figure 3.4.

```
=== DISAMBIGUATION TRIGGERED ===
line_row = 12, has_ambiguous = 1
Calling disambiguate_candidates with 10 words...
Running context-aware disambiguation (may take 15-30 sec on first run)...
JSON saved to: C:\Users\leian\AppData\Local\Temp\mabaybay_candidates.json
Candidates JSON: [{"mayroon"}, {"kaming"}, {"solosyong". "solosyung"}, {"asido", "asero"}, {"
Python status: 0
Python output: [mayroon kaming solusyong asido para sa paglilinis ng mga baldosa
]
Disambiguation returned: mayroon kaming solusyong asido para sa paglilinis ng mga baldosa
Elapsed time is 169.962671 seconds.
:>>
```

Figure 3.4. Disambiguation Integrated into MaBaybay-OCR

This modular design preserves the original OCR pipeline while introducing a post-processing layer without requiring modifications to the original MATLAB codebase. The complete implementation of the disambiguation module consists of two main components:

BaybayinDisambiguator Class (`src/disambiguator.py`)

This component serves as the core engine responsible for scoring and candidate selection. Upon initialization, it loads (1) the RoBERTa-Tagalog model and (2) corpus statistics derived from three Filipino text sources: Tagalog literary texts, religious texts, and news corpora. The full source code for this class is presented in Appendix B.1.

The `disambiguate()` method processes the candidate list on a per-position basis. For ambiguous positions (i.e., positions with multiple candidates), it computes MLM-PLL scores for each candidate and selects the highest-scoring option. A code snippet illustrating this scoring process is in Figure 3.5.


```

# For each ambiguous position, compute scores and select best candidate:

# Get MLM scores for all candidates
mlm_scores = self._get_mlm_scores(mlm_words, pos, item)

# Score all candidates with MLM-based semantic feature
scores = {
    c: self.score_candidate(c, mlm_score=mlm_scores[c])
    for c in item
}

# Select best candidate
best = max(scores.keys(), key=lambda c: scores[c]['combined'])
result.append(best)

```

Figure 3.5. Code Snippet of the Scoring Process

Wrapper Script (disambiguate.py)

Complementing the core engine, a wrapper script was implemented to serve as the interface between the OCR output and the Python environment. This script handles candidate normalization, detects context availability to assign appropriate scoring weights, and suppresses verbose model-loading logs to maintain a clean output stream for the MATLAB console.

The wrapper encapsulates all MATLAB integration logic, allowing the core `BaybayinDisambiguator` class to remain implementation-agnostic.

3.3.2 Pseudo-Log-Likelihood (PLL) Scoring for Context-Aware Disambiguation

The semantic context feature was implemented using the Masked Language Model (MLM) head of the RoBERTa architecture. This approach evaluated how well each candidate word fit within the sentence context by measuring the probability of the candidate under the RoBERTa-Tagalog base language model developed by Cruz and Cheng (2022).

3.3.2.1 PLL Computation

The core PLL computation was implemented using the algorithm described by Salazar et al. (2020) to leverage the bidirectional contextual understanding of the Tagalog language. Although the Baybayin script is written and read from left to right, the semantic and grammatical dependencies required for disambiguation are inherently bidirectional, as the correct transliteration of a word often depends on both its preceding and succeeding words.

For each ambiguous word position, a tokenized sentence was generated with the target position masked. Because the RoBERTa-Tagalog tokenizer utilizes Byte-Pair Encoding (BPE), the scoring logic was designed to mask each subtoken of the candidate word individually. The mathematical formulation of this scoring process is presented in Figure 3.6.

$$PLL(\text{candidate}) = \sum_{i=1}^k \log P(t_i | \text{masked sentence at position } p_i)$$

Figure 3.6. Mathematical Equation for Candidate Pseudo-Log-Likelihood (PLL) Computation

In Figure 3.6, t_i represents the original token at position p_i , while k denotes the total number of subtokens. This summation ensures that multi-token words are not unfairly penalized relative to single-token words during the candidate selection process. The complete code implementation of this is provided in Appendix C.1.

3.3.2.2 MLM Score Normalization

To convert the raw PLL scores into a probabilistic ranking, the implementation utilized a softmax normalization function. This allowed the system to produce scores between 0 and 1 that sum to 1 within the candidate set. The transfor-

mation of these raw values into a normalized distribution was performed using the equation shown in Figure 3.7.

$$\text{MLM score} = \frac{\exp(PLL_i)}{\sum_j \exp(PLL_j)}$$

Figure 3.7. Equation for Softmax Normalization of Masked Language Model Scores.

This normalization approach provided a substantially strong semantic signal, as it interrogated the model’s knowledge of contextual appropriateness without a left-to-right bias. The complete code implementation of equation is provided in Appendix C.2.

3.3.3 Frequency-Based Scoring for No-Context Scenarios

A frequency-based scoring mechanism was implemented to serve as a fallback for single-word inputs where no surrounding sentence context was available.

3.3.3.1 Corpus Collection

The statistics used for this method were computed during the initialization phase from three Filipino text sources: Tagalog literary texts, Tagalog religious texts, and Tagalog news texts. The Tagalog literary and religious corpora were obtained from the Philippine Language Corpora compiled by Dita et al. (2008), while the Tagalog news texts were sourced from the dataset used by Bunag and Esquivel (2023). This diverse collection ensured that the statistical prior was informed by both formal and contemporary usage. The loading process involved processing the text sentence by sentence, extracting words via regular expressions, and computing both word-level and bigram frequencies.

3.3.3.2 Frequency Score Implementation

The frequency score for a given word w was implemented using logarithmic normalization to prevent high-frequency words from disproportionately dominating the overall scores. The logarithmic normalization applied to these counts is expressed in the formula shown in Figure 3.8.

$$\text{Frequency Score}(w) = \frac{\log(\text{count}(w) + 1)}{\log(\text{max_frequency} + 1)}$$

Figure 3.8. Formula for the Log-Normalized Frequency Scoring Mechanism

As seen in Figure 3.8, $\text{count}(w)$ represents the occurrence count of the word w within the corpus, while max_frequency denotes the frequency count of the most frequently occurring word. Words that did not appear in the corpus were assigned a small non-zero score of 0.1 to provide graceful degradation for out-of-vocabulary words. The complete code implementation of formula is provided in Appendix D.1.

3.3.4 Baybayin Ambiguous Pairs

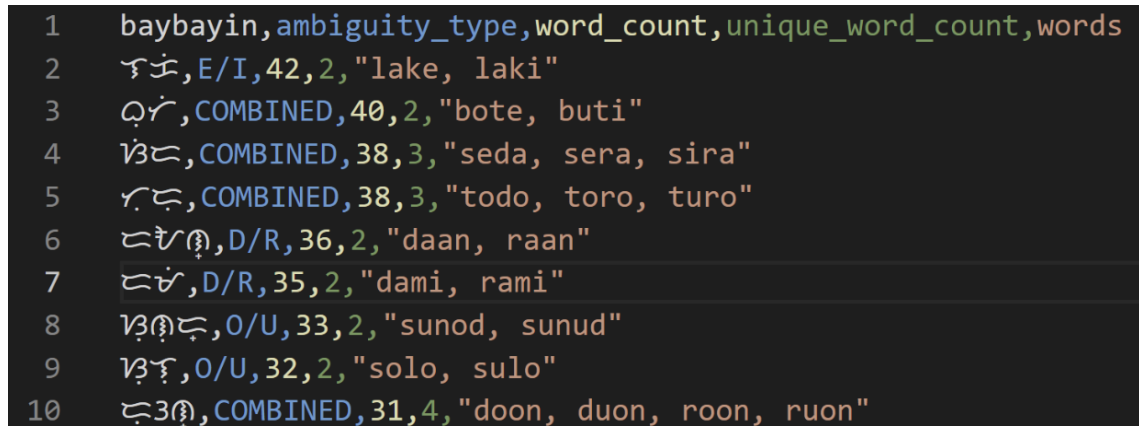
The identification of ambiguous word pairs was conducted through a systematic analysis of a MaBaybay Filipino word corpus containing over 74,419+ words. The goal of this process is to detect words that share identical Baybayin representations despite differences in their Latin orthographic forms.

Each word in the corpus was first converted into its Baybayin equivalent using a greedy left-to-right matching approach, prioritizing longer character sequences (e.g., “nga”) to preserve phonetic accuracy. During this conversion, selected phonetic distinctions—specifically e/i, o/u, and d/r—were intentionally collapsed to expose potential ambiguity.

Following conversion, words were grouped according to their resulting Bay-

bayin representations. Groups containing multiple distinct Latin words were identified as ambiguous candidates. These candidates were then categorized based on the type of phonetic ambiguity present, namely: **E/I**, **O/U**, **D/R**, **COMBINED**, or **UNKNOWN**.

To ensure dataset quality, the resulting groups underwent a filtering process that removed duplicate entries, excluded proper-noun-only words, and eliminated non-dictionary terms. The finalized output was stored in a structured dataset (`ambiguous_pairs_complete.csv`), as shown in Figure 3.9, which contains each ambiguous pair together with its corresponding Baybayin representation, ambiguity classification, and frequency statistics.



1	baybayin	ambiguity_type	word_count	unique_word_count	words
2	ᜏᜒᜓ	E/I	42	2	"lake, laki"
3	ᜏᜒᜓ	COMBINED	40	2	"bote, buti"
4	ᜏᜒᜓ	COMBINED	38	3	"seda, sera, sira"
5	ᜏᜒᜓ	COMBINED	38	3	"todo, toro, turo"
6	ᜏᜒᜓ	D/R	36	2	"daan, raan"
7	ᜏᜒᜓ	D/R	35	2	"dami, rami"
8	ᜏᜒᜓ	O/U	33	2	"sunod, sunud"
9	ᜏᜒᜓ	O/U	32	2	"solo, sulo"
10	ᜏᜒᜓ	COMBINED	31	4	"doon, duon, roon, ruon"

Figure 3.9. Sample Structure of the Finalized Ambiguous Pair Dataset

From this dataset, the 15 most known ambiguous pairs were manually selected for use in evaluation. These include examples such as asero/asido, bote/buti, mesa/misa, and hito/heto, etc ensuring representation across multiple ambiguity categories.

To support the annotation and validation process, a dedicated web-based interface was developed. The tool enables efficient browsing of ambiguous pairs and facilitates the tracking of sentence-level annotations, as shown in Figure 3.10.

Baybayin Dataset Manager

Target: 50 sentences per "sense" (meaning) for your **Gold-Standard Dataset**.

Gemini API Key

Add New Ambiguous Group

 Add Group

Active Ambiguity Group

Delete Group

Total Sentences: 0

bote: 0/50

buti: 0/50

Add Sentence (Manual or AI)

Save Sentence
AI Assist (Gemini 2.5 Flash)
Download .txt

Figure 3.10. Web-Based Interface for Ambiguous Pair Annotation and Validation

3.3.5 Standardized Evaluation Dataset

To ensure consistent and reproducible evaluation of the disambiguation system, a standardized evaluation dataset was constructed and organized within the `gold_standard_dataset` directory. This dataset serves as the primary benchmark for assessing model performance against manually validated ground truth.

The dataset is composed of three main components: annotated sentences, corresponding Baybayin image inputs, and recorded evaluation outputs.

The sentences component contains ground truth data consisting of manually constructed and annotated sentences. Each sentence is grouped according to its associated ambiguous word pair, with explicit labeling of the correct word interpretation. These annotations serve as the reference standard for evaluating disambiguation accuracy.

The images component contains Baybayin-rendered text corresponding to each ambiguous pair. These images are organized into subdirectories and rep-

resent the actual input to the OCR system. The inclusion of `tagalog` stylized font ensures that the evaluation reflects realistic OCR conditions. A sample Baybayin-rendered input image is shown in Figure 3.11.

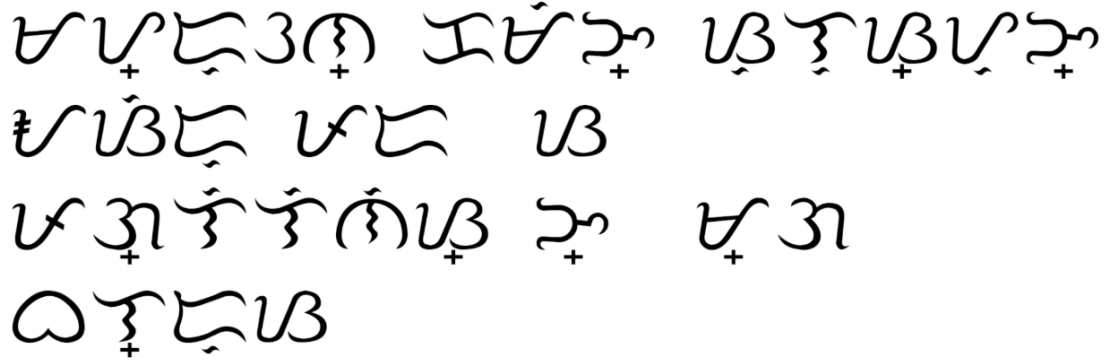


Figure 3.11. Sample Baybayin-rendered OCR input for the asero/asido pair.

The results component stores the outputs generated by the disambiguation method when applied to the dataset. This includes predicted word interpretations and computed performance metrics, enabling systematic comparison against the baseline heuristic.

Overall, the `gold_standard_dataset` provides a controlled and reliable benchmark for evaluating the proposed MLM-PLL disambiguation approach, ensuring that performance measurements are consistent, traceable, and comparable across experimental conditions.

3.4 Testing

Testing was conducted using the standardized gold standard dataset, which consists of 15 manually curated ambiguous word pairs. Test cases were organized using a structured JSON format to ensure consistency in input representation and facilitate systematic execution of test scripts. The structure of the test data is illustrated in Figure 3.12.

```

{
  'sentence': str,           # Ground truth sentence with target word
  'ground_truth': str,      # Original sentence (for evaluation reference)
  'ocr_candidates': [       # OCR candidate words (one per position)
    'word1',                # Unambiguous word (single candidate)
    ['word_a', 'word_b'],   # Ambiguous word (multiple candidates)
    'word3',                # ...
  ]
}

```

Figure 3.12. JSON structure used for representing transliteration test cases

Each test script independently executes a standardized testing pipeline consisting of the following steps:

1. Ground-truth sentences corresponding to each ambiguous pair are loaded from the `gold_standard_dataset/sentences/` directory.
2. Synthetic OCR candidate lists are generated, wherein the target word position is intentionally marked as ambiguous by including the two candidate interpretations.
3. The `BaybayinDisambiguator` is applied to the generated test inputs.
4. System outputs are recorded for each test instance.
5. The resulting predictions are stored as structured JSON files within the `gold_standard_dataset/results/` directory, following the naming convention `results_{pair}.json`.

For example, in testing the *asero/asido* pair, both candidate words were explicitly assigned to the target position within each sentence while the surrounding contextual words remained fixed. This approach ensures that the evaluation directly measures the system’s ability to resolve ambiguity under controlled conditions. A sample test script used is provided in Appendix F.1.

3.5 Evaluation

The performance of the proposed disambiguation system was evaluated using the outputs generated during the testing phase. Evaluation was performed by comparing the system’s predicted word interpretations against the manually annotated ground truth for each test instance in the gold standard dataset.

The primary evaluation metric used in this study is disambiguation accuracy, defined as the proportion of correctly predicted word interpretations over the total number of evaluated instances. This metric provides a direct measure of the system’s effectiveness in resolving Baybayin transliteration ambiguity.

Accuracy was computed per ambiguous pair, with each pair consisting of 100 test sentences (50 instances per variant word) to ensure balanced evaluation. The overall system accuracy was then obtained by aggregating results across all 15 ambiguous pairs.

To provide a baseline for comparison, the system was evaluated against the default MaBaybay-OCR behavior, which selects the first candidate without contextual disambiguation. This comparison quantifies the improvement achieved through the proposed context-aware disambiguation approach. Finally, runtime measurements were considered to assess the computational efficiency of the system during evaluation. These evaluation metrics—disambiguation accuracy and runtime performance—are consistent with the methodology employed in *bAI-bAI* of Bernardo and Estuar (2025). The results of these analyses are presented in Chapter 4.

Chapter 4

Results and Discussion

This chapter presents the results and discussion of the experiments conducted to evaluate the context-aware Baybayin transliteration system. The findings are organized into two primary sections: the disambiguation accuracy across ambiguous pairs and the computational performance of the system.

Accuracy

The performance of the proposed MaBaybay-OCR 2.0 system was measured against the baseline MaBaybay-OCR, which relies on a first-candidate selection heuristic. The evaluation was conducted using a standardized dataset comprising 1,600 samples designed to test e/i, o/u, and d/r ambiguities. Table 4.1 summarizes the resulting accuracies obtained from the experimentation.

Table 4.1. Resulting Accuracies per Method

Method	Correct Samples	Accuracy
Baseline (first candidate)	733/1600	46%
MLM-PLL	1325/1600	83%

As shown in Table 4.1, the baseline method achieved an accuracy of only 46%. This result was consistent with the theoretical expectation of approximately 50% for a balanced dataset where the system lacks a mechanism to

choose between two equally likely phonetic candidates. Because the baseline system always selects the first available candidate in the transliteration mapping, it essentially fails to resolve any instance where the second or third candidate is the linguistically correct choice.

In contrast, the MLM-PLL approach yielded a significantly higher accuracy of 83%. This 37% improvement demonstrated that the disambiguation layer successfully utilized the bidirectional context of the surrounding Tagalog words to identify the correct transliteration. By calculating the Pseudo-Log-Likelihood (PLL) of each candidate, the system was able to override the default heuristic when the semantic environment favored a different word—for instance, choosing *bote* (bottle) over *buti* (goodness) when the sentence contained the word *tubig* (water).

Computational Performance

The introduction of a deep learning-based disambiguation layer necessitated an analysis of the computational overhead. The runtime was recorded per sample to evaluate the system’s suitability for real-time applications. Table 4.2 presents the average runtimes for one sample across the different methods.

Table 4.2. Runtimes per method for one sample

Method	Runtime
Baseline (first candidate)	0 ms
MLM-PLL	237 ms

According to Table 4.2, the baseline method incurred 0 ms of additional processing time as it merely selected the first string in a pre-defined list. The proposed MLM-PLL system required an average of 237 ms to process a single sample. While this represents a measurable increase in latency, the performance remained within a practical range for digital archival and educational

tools.

When compared to previous literature, this runtime highlights the efficiency of the RoBERTa-based rescoring approach. Experimental Large Language Model (LLM) methods, such as those utilizing GPT-4o mini in the bAI-bAI system, have been reported to take as long as 3280 ms per sample. The 237 ms runtime observed in this study confirms that the use of a smaller, Tagalog-specific masked language model provides a viable middle ground—offering a speed increase over generative LLMs while maintaining high transliteration accuracy.

The results indicated that the context-aware disambiguation module successfully addressed the primary research problem. The jump from 46% to 83% accuracy suggests that most e/i, o/u, and d/r ambiguities are not merely phonetic but are semantically distinct enough for a transformer model to resolve. The 17% error rate that remained was attributed to cases of high-level semantic ambiguity where multiple transliterations could remain grammatically plausible, or where the input text contained colloquialisms outside the training domain of the RoBERTa-Tagalog model. Furthermore, the statistical fallback mechanism was observed to be effective in preventing nonsensical outputs for isolated words. By incorporating frequency-based priors, the system was able to prioritize common words like doktor over non-dictionary fragments like doktod in cases where no sentence context was available to the MLM. Overall, the MaBaybay-OCR 2.0 system achieved its goal of providing a robust, linguistically accurate transliteration of the Baybayin script with manageable computational costs.

Chapter 5

Conclusion and Future Work

This study successfully developed the context-aware MaBaybay-OCR 2.0 system, aimed at advancing the digital preservation and linguistic accuracy of the Baybayin script. A disambiguation module was designed to address the structural overlaps in the script by integrating a pre-trained RoBERTa-Tagalog model and frequency-based scoring. The resulting system demonstrated a significant capacity to resolve the “one-to-many” mapping problem inherent in Baybayin, specifically regarding the interchangeable use of *e/i*, *o/u*, and *d/r*. The end-to-end context-aware system achieved a transliteration accuracy of 83% on a standardized dataset of 1,600 samples, representing a substantial 37-percentage-point improvement over the baseline accuracy of 46%. This indicates that the system is robust in using bidirectional sentential context to select the most grammatically appropriate transliteration, thereby reducing the semantic distortion frequently encountered in literal, heuristic-based OCR systems.

Among the evaluated computational approaches, the use of Masked Language Model (MLM) scoring via Pseudo-Log-Likelihood (PLL) emerged as the most effective method for balancing performance and speed. The system achieved a runtime of 237 ms per sample, which is significantly more efficient than previous generative Large Language Model (LLM) approaches that have been reported to take upwards of 3,280 ms. This confirms that for low-resource scripts like Baybayin, specialized bidirectional transformers like RoBERTa-Tagalog pro-

vide a superior middle ground—offering the deep linguistic context required for disambiguation without the prohibitive computational latency of larger generative models. Furthermore, the implementation of log-normalized frequency statistics scoring proved essential for resolving orthographic ambiguities in isolated, zero-context words, such as correcting *doktod* to *doktor* based on lexical prevalence in Tagalog corpora.

Finally, the construction of a standardized evaluation dataset—the first of its kind specifically for Baybayin lexical disambiguation—represents a foundational contribution to the research community. By curating 50 sentences per “sense” for 15 common ambiguous pairs, the study provided a rigorous benchmark to prove that the system’s decisions were grounded in linguistic patterns rather than random chance. The findings collectively suggest that a hybrid approach—combining deep learning for semantic context and statistical priors for lexical validation—establishes a highly robust framework for the accurate digital restoration of Philippine historical scripts.

Future Work

While this research has successfully addressed one of the most significant challenges in the field—applying NLP algorithms to determine the most appropriate Latin transliteration—it fulfills only one part of the broader technological roadmap for Baybayin. The original MaBaybay-OCR study identified several critical research opportunities that remain unaddressed and should be prioritized in future work. Foremost among these is the inclusion of Baybayin punctuation marks, as traditional and modern texts utilize specific danda-like symbols that are currently excluded from most OCR systems. Incorporating these symbols would enhance the structural accuracy of digitized manuscripts and bring the system closer to a full-featured archival tool.

Additionally, future efforts should build upon the current system to develop a comprehensive two-way Baybayin-Latin transliterator. While this study focused

on the Baybayin-to-Latin path, creating a robust system that can convert Latin Filipino text into accurate Baybayin representation would enable a more complete cultural exchange and support educational efforts. Beyond these goals, the RoBERTa-Tagalog model itself can be refined by fine-tuning the underlying model on “Taglish” code-switching and modern colloquialisms, which currently limit contextual understanding. Lastly, extending the software to other platforms, such as web-based interfaces and mobile applications, remains a critical goal for increasing the script’s accessibility to the general public.

References

- Balbutin, A. J. (2023). Baybayin in public places and shops.
- Bayani Art (2015). Baybayin – ancient writing script of the philippines.
- Bernardo, J. S. and Estuar, M. R. J. E. (2025). bai-bai: A context-aware transliteration system for baybayin scripts. In *Proceedings of the Second Workshop in South East Asian Language Processing*, pages 1–9.
- Bunag, K. L. T. and Esquivel, R. A. (2023). Transformer-based conditional language models to generate filipino news articles. In *Proceedings of the International Conference on Industrial Engineering and Operations Management*, pages 2231–2237. IEOM Society International.
- Camba, A. T. (2021). *Baybayin: The Role of a Written Language in the Cultural Identity and Socio Psychological Well-Being of Filipinos*. Ph.d. thesis, Harvard University.
- Cruz, J. C. B. and Cheng, C. (2022). Improving large-scale language models and resources for filipino. In *Proceedings of the 13th Conference on Language Resources and Evaluation (LREC 2022)*, pages 6548–6555. European Language Resources Association.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186.
- Dita, S., Roxas, R. E., and Inventado, P. (2008). Building online corpora of

- philippine languages. In *Proceedings of the 23rd Pacific Asia Conference on Language, Information and Computation*.
- Kauf, C. and Ivanova, A. A. (2023). A better way to do masked language model scoring. *arXiv preprint arXiv:2305.10588*.
- Kissos, I. and Dershowitz, N. (2016). Ocr error correction using character correction and feature-based word classification. In *2016 IEEE International Conference on the Science of Electrical Engineering (ICSEE)*.
- Kuya Bai (2022). What is baybayin?
- Ligsay, A. M. P., Rivera, J. B., and Villaverde, J. F. (2022). Optical character recognition of baybayin writing system using yolov3 algorithm. In *2022 IEEE International Conference on Artificial Intelligence in Engineering and Technology (IICALET)*, pages 1–5. IEEE.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. (2019). Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.
- Lu, B. J. (2023). Preserving our cultural heritage: The revival of baybayin.
- Narra Studio (2019). Baybayin: The ancient filipino script lives on.
- Pino, R., Mendoza, R., and Sambayan, R. (2021). A baybayin word recognition system. *PeerJ Computer Science*, 7:e596.
- Pino, R., Mendoza, R., and Sambayan, R. (2025). Mababay-ocr: A matlab-based baybayin optical character recognition package. *SoftwareX*, 29:102003.
- Salazar, J., Liang, D., Nguyen, T. Q., and Kirchhoff, K. (2020). Masked language model scoring. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 2699–2712. Association for Computational Linguistics.
- Schachter, P. and Reid, L. A. (2018). Tagalog. In Comrie, B., editor, *The World's Major Languages*, page 25. Routledge, 3 edition.
- Stone, R. K. (2025). Reviving baybayin: The pre-hispanic writing system of the

philippines, and raising awareness in the united states.

Tennant, A. (2024). Baybayin: A written cultural revival.

University of Santo Tomas (2021). The baybayin documents.

Vilvar, R. A., Hammond, D. S. C., Santos, F. M. R., and Alar, H. S. (2022).

Baybayin script word recognition and transliteration using a convolutional neural network. *SSRN Electronic Journal*.

Appendices

Appendix A

MaBayBay-OCR Console

This appendix presents the console-based implementation of the MaBayBay-OCR system. Figure A.1 illustrates the command-line interface used to execute the OCR and transliteration process, while Figure A.2 presents a sample output produced by the system.

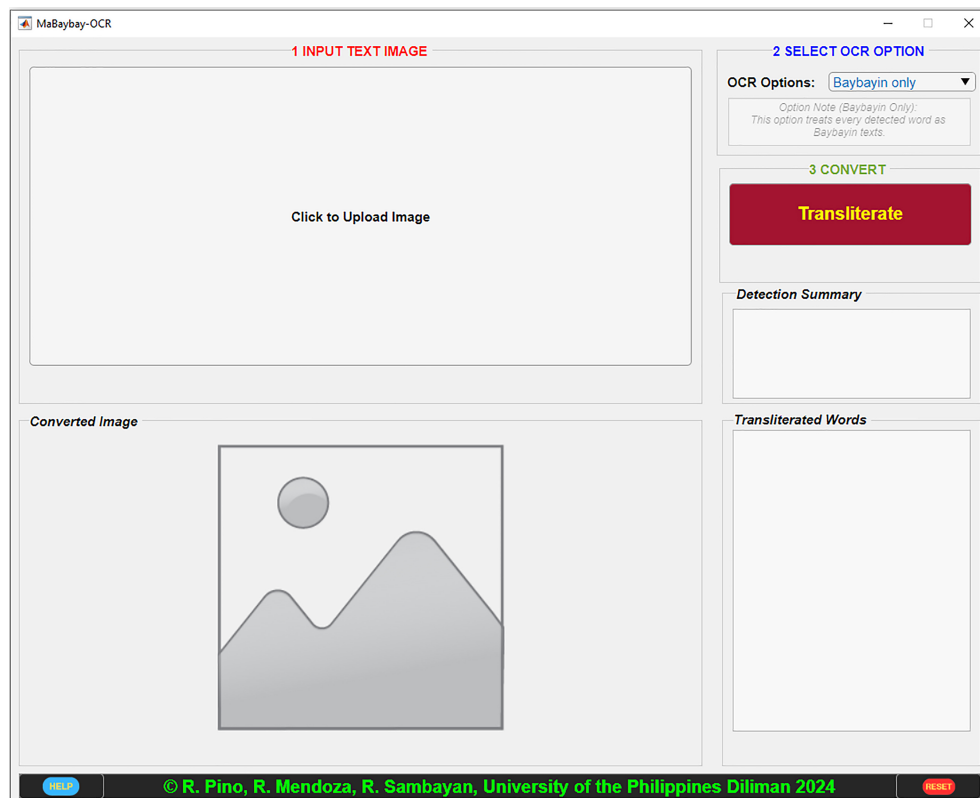


Figure A.1. The MaBayBay-OCR Interface

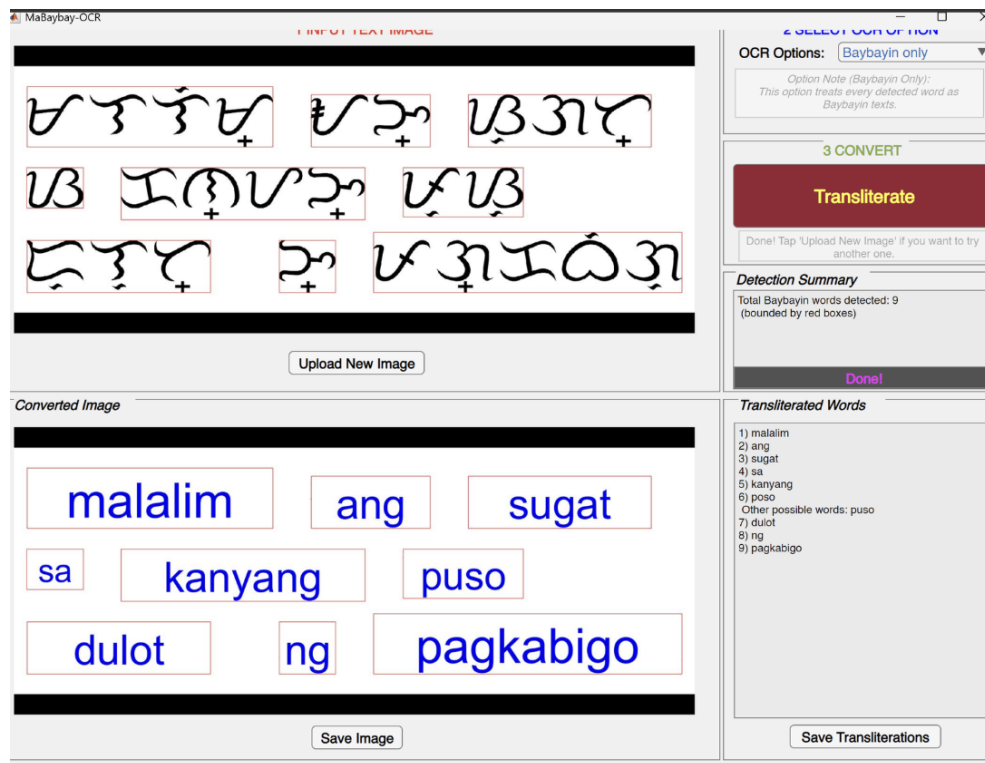


Figure A.2. Sample output of the system

Appendix B

BaybayinDisambiguator Class

This appendix presents the complete implementation of the BaybayinDisambiguator class, which functioned as the core engine for linguistic scoring. Figure B.1 shows the initialization logic where the pre-trained weights for the RoBERTa-Tagalog model and corpus statistics were loaded. The default configuration is provided in Figure B.2.

```

class BaybayinDisambiguator:
    def __init__(
        self,
        model_name: str = "jcblaise/roberta-tagalog-base",
        corpus_files: List[str] = None,
        exclude_sentences: List[str] = None,
        weights: Dict[str, float] = None,
        device: str = None
    ):
        self.device = device or ("cuda" if torch.cuda.is_available()
else "cpu")
        self.weights = weights or DEFAULT_WEIGHTS
        corpus_files = corpus_files or DEFAULT_CORPORA

        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model =
AutoModelForMaskedLM.from_pretrained(model_name).to(self.device)
        self.model.eval()

        self.corpus = CorpusStatistics(
            text_files=corpus_files,
            exclude_sentences=exclude_sentences
        )

```

Figure B.1. Transformer Model Initialization

```

# Default configuration
DEFAULT_MODEL = "jcbldaise/roberta-tagalog-base"

# Weights for ambiguous words: MLM-PLL only (context-aware)
DEFAULT_WEIGHTS = {
    'semantic': 1.0,      # MLM PLL scoring (context-aware, the primary
                          # signal)
    'frequency': 0.0,    # Not used for ambiguous words with context
    'cooccurrence': 0.0, # Not used for ambiguous words with context
    'morphology': 0.0     # Not used for ambiguous words with context
}

# For unambiguous words: frequency-only scoring
FREQUENCY_ONLY_WEIGHTS = {
    'semantic': 0.0,
    'frequency': 1.0,
    'cooccurrence': 0.0,
    'morphology': 0.0
}

# Text corpora paths
DEFAULT_CORPORA = [
    "corpus/Tagalog_Literary_Text.txt",
    "corpus/Tagalog_Religious_Text.txt",
    "corpus/Tagalog_Balita_Texts_Balanced.txt"
]

```

Figure B.2. Default Configuration

Appendix C

MLM-PLL Code

This appendix presents the complete implementation of the PLL scoring mechanism, which measures how well a candidate word fits within a given context by masking each of its subtokens one at a time and summing their corresponding log-probabilities. Figure C.1 shows the implementation logic of the PLL scoring procedure, while Figure C.2 presents the MLM score normalization process.

```

def _get_candidate_pll(
    self,
    sentence_words: List[str],
    position: int,
    candidate: str
) -> float:
    words = list(sentence_words)
    words[position] = candidate
    full_sentence = ' '.join(words)

    encoding = self.tokenizer(
        full_sentence,
        return_tensors='pt',
        truncation=True,
        max_length=128
    )
    token_ids = encoding['input_ids'][0].clone().to(self.device)
    attention_mask = encoding['attention_mask'][0].to(self.device)

    cand_tokens = self.tokenizer.encode(' ' + candidate,
add_special_tokens=False)
    cand_positions = self._find_subtoken_positions(token_ids.cpu(),
cand_tokens)

    if not cand_positions:

```

```

    # Find candidate's subtoken positions
    cand_tokens = self.tokenizer.encode(' ' + candidate,
add_special_tokens=False)
    cand_positions = self._find_subtoken_positions(token_ids.cpu(),
cand_tokens)

    if not cand_positions:
        # Try without space prefix (for sentence-initial words)
        cand_tokens = self.tokenizer.encode(candidate,
add_special_tokens=False)
        cand_positions = self._find_subtoken_positions(token_ids.cpu(),
cand_tokens)

    if not cand_positions:
        return -100.0 # Very low score for unfound words

    # Compute PLL: mask each subtoken one at a time, sum log-probs
    total_log_prob = 0.0

    for pos in cand_positions:
        masked = token_ids.clone()
        original_token = masked[pos].item()
        masked[pos] = self.tokenizer.mask_token_id

        with torch.no_grad():
            outputs = self.model(
                masked.unsqueeze(0),
                attention_mask=attention_mask.unsqueeze(0)
            )
            logits = outputs.logits[0, pos]
            log_probs = torch.log_softmax(logits, dim=-1)
            total_log_prob += log_probs[original_token].item()

    return total_log_prob

```

Figure C.1. Core PLL Computation

```

def _get_mlm_scores(
    self,
    sentence_words: List[str],
    position: int,
    candidates: List[str]
) -> Dict[str, float]:
    pll_scores = {}
    for candidate in candidates:
        pll_scores[candidate] = self._get_candidate_pll(
            sentence_words, position, candidate
        )

    max_pll = max(pll_scores.values())
    exp_scores = {
        c: math.exp(s - max_pll)
        for c, s in pll_scores.items()
    }
    total = sum(exp_scores.values())

    if total > 0:
        return {c: exp_scores[c] / total for c in candidates}
    else:
        return {c: 1.0 / len(candidates) for c in candidates}

```

Figure C.2. MLM Score Normalization

Appendix D

Frequency-Based Code

This appendix presents the code for calculating log-normalized scores, implemented through the specialized Python method displayed in Figure D.1.

```
def get_frequency_score(self, word: str) -> float:
    count = self.word_freq.get(word.lower(), 0)

    if count == 0:
        return 0.1

    max_freq = self.word_freq.most_common(1)[0][1]
    return min(1.0, np.log(count + 1) / np.log(max_freq + 1))
```

Figure D.1. Frequency Scoring

Appendix E

System Integration

This appendix details the the implementation of the system-call mechanism that allows the Python-based `BaybayinDisambiguator` class to be invoked during the MaBaybay-OCR execution pipeline is shown in Figure E.1.

```

% =====
% disambiguate_candidates.m
% Use Python disambiguation model instead of defaulting to first
candidate
% =====

function result = disambiguate_candidates(transliterations)
    thesis_dir = fullfile(fileparts(mfilename('fullpath')), '..', '..');
    python_script = fullfile(thesis_dir, 'disambiguate.py');
    python_exe = fullfile(thesis_dir, '.venv', 'Scripts', 'python.exe');
    temp_json = fullfile(tempdir, 'mabaybay_candidates.json');

    candidates = cell(1, length(transliterations));
    for i = 1:length(transliterations)
        word_cands = transliterations{i};
        if ischar(word_cands)
            candidates{i} = {word_cands};
        elseif iscell(word_cands)
            candidates{i} = word_cands(:)';
        else
            candidates{i} = {char(word_cands)};
        end
    end

    json_str = jsonencode(candidates);
    fid = fopen(temp_json, 'w');
    fprintf(fid, '%s', json_str);
    fclose(fid);

    fprintf('Running context-aware disambiguation (may take 15-30 sec on
first run)...\n');
    fprintf('JSON saved to: %s\n', temp_json);
    fprintf('Candidates JSON: %s\n', json_str);
    python_cmd = sprintf('%s "%s" "%s"', python_exe, python_script,
temp_json);
    [status, output] = system(python_cmd);

```

```

fprintf('Python status: %d\n', status);
fprintf('Python output: [%s]\n', output);

if status ~= 0
    warning('Disambiguation failed, using first candidates. Error:
%s', output);
    result = cell(1, length(transliterations));
    for i = 1:length(transliterations)
        word_cands = transliterations{i};
        if iscell(word_cands)
            result{i} = word_cands{1};
        else
            result{i} = char(word_cands);
        end
    end
    return;
end

output = strtrim(output);
result = strsplit(output, ' ');

delete(temp_json);
end

```

Figure E.1. MATLAB Integration Code

Appendix F

Testing Framework

This appendix presents the automation scripts used in evaluating the context-aware system. A full sample source code snippet is shown in Figure F.1.

```

"""
Test disambiguator on word pairs
"""

import json
import re
import sys
import os
import time
from pathlib import Path

PROJECT_ROOT = str(Path(__file__).parent.parent)
os.chdir(PROJECT_ROOT)
sys.path.insert(0, PROJECT_ROOT)

from src.disambiguator import BaybayinDisambiguator

def get_clean_words(sentence):
    words = re.findall(r'\b\w+\b', sentence.lower())
    return words

def parse_sentence_file(filepath):
    bote_sentences = []
    buti_sentences = []

    with open(filepath, "r", encoding="utf-8") as f:
        lines = [line.strip() for line in f if line.strip()]

    for line in lines:
        words = get_clean_words(line)
        if "bote" in words:
            bote_sentences.append(line)
        elif "buti" in words:
            buti_sentences.append(line)

    return bote_sentences, buti_sentences

```

```

SENTENCE_FILE = "gold_standard_dataset/sentences/03_bote_buti.txt"
bote_sentences, buti_sentences = parse_sentence_file(SENTENCE_FILE)

test_data = []

for sent in bote_sentences:
    words = sent.split()
    candidates = []

    for word in words:
        clean_word = re.sub(r'^\w', '', word.lower())
        if clean_word == "bote":
            candidates.append(["bote", "buti"])
        else:
            candidates.append(clean_word)

    test_data.append({
        'ground_truth': sent,
        'ocr_candidates': candidates
    })

for sent in buti_sentences:
    words = sent.split()
    candidates = []

    for word in words:
        clean_word = re.sub(r'^\w', '', word.lower())
        if clean_word == "buti":
            candidates.append(["bote", "buti"])
        else:
            candidates.append(clean_word)

    test_data.append({
        'ground_truth': sent,
        'ocr_candidates': candidates
    })

```

```

print(f"{'='*70}")
print("BASELINE: MaBaybay Default (First Candidate)")
print(f"{'='*70}")

baseline_correct_total = 0
baseline_correct_bote = 0
baseline_correct_buti = 0

baseline_start_time = time.time()
for test_item in test_data:
    gt = test_item['ground_truth']
    gt_words = get_clean_words(gt)

    if "bote" in gt_words:
        baseline_correct_total += 1
        baseline_correct_bote += 1

baseline_time = time.time() - baseline_start_time
baseline_accuracy = baseline_correct_total / 100 * 100

print(f"Bote accuracy: {baseline_correct_bote}/50 = {baseline_correct_bote/50:.2%}")
print(f"Buti accuracy: {baseline_correct_buti}/50 = {baseline_correct_buti/50:.2%}")
print(f"Overall accuracy: {baseline_correct_total}/100 = {baseline_accuracy:.2f}%")
print(f"Execution time: {baseline_time*1000:.2f}ms ({(baseline_time/100)*1000:.4f}ms per position)")

print(f"\n{'='*70}")
print("INITIALIZING MODEL")
print(f"{'='*70}")

all_test_sentences = [item['ground_truth'] for item in test_data]
init_start_time = time.time()
model = BaybayinDisambiguator(
    corpus_files=[

```

```

        "corpus/Tagalog_Literary_Text.txt",
        "corpus/Tagalog_Religious_Text.txt",
        "corpus/Tagalog_Balita_Texts_Balanced.txt"
    ],
    exclude_sentences=all_test_sentences
)
init_time = time.time() - init_start_time
print(f"Initialization time: {init_time:.2f}s")

print(f"\n{'='*70}")
print("CONTEXT-AWARE DISAMBIGUATION (Pure MLM-PLL)")
print(f"{'='*70}")

eval_start_time = time.time()
metrics, results = model.evaluate(test_data, show_progress=True,
use_mlm=True)
eval_time = time.time() - eval_start_time

print(f"\nAmbiguous words (bote/buti): {metrics['total_ambiguous']}")
print(f"Correct disambiguations: {metrics['correct_ambiguous']}")
print(f"★ Context-aware accuracy: {metrics['ambiguous_accuracy']:.2%}
★")
print(f"Evaluation time: {eval_time:.2f}s
({(eval_time/metrics['total_ambiguous'])*1000:.2f}ms per ambiguity)")

```

Figure F.1. Sample Test Script

Appendix G

Ambiguous Words

This appendix presents the list of ambiguous Filipino word pairs and triplets used in constructing the evaluation dataset for the context-aware disambiguation system. A full list of the selected ambiguous terms is shown in Table G.1.

Table G.1. List of Ambiguous Word Pairs Used in the Dataset

Pair	Word 1	Word 2	Word 3
1	asero = steel	asido = acid	
2	bola = ball	bula = bubble	
3	bote = bottle	buti = goodness	
4	boto = vote	buto = bone	
5	higante = giant	higanti = revenge	
6	heto = here it is	hito = catfish	
7	itodo = go all out	ituro = point	
8	kometa = comet	kumita = to earn	
9	kompas = compass	kumpas = wave	
10	mesa = table	misa = mass	
11	polo = polo shirt	pulo = island	
12	poso = well (water)	puso = heart	
13	tela = cloth/fabric	tila = seems like	
14	todo = full force	toro = bull	turo = teaching
15	toyo = soy sauce	tuyo = dried fish/dry	